Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
ARTIFICIAL INTELLIGENCE

Artificial Intelligence Application Development Practical

AI-Based Diabetes Prediction System

Student Name: Siddhesh Samadhan Kamble

Roll Number: T009

Class: B.Sc. Information Technology

Division: A

Subject: Artificial Intelligence Application Development Practical

Academic Year: 2026–2027

College Name: SHETH L.U.J & SIR M.V COLLAGE

Department Name: Information Technology

Faculty Name: Ms. Chetana Tanavade

1. Introduction :

The AI-Based Diabetes Prediction System is a machine-learning-based web application developed to predict the outcome of diabetes-related health data. The system accepts eight input parameters—Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function and Age—and uses a trained Logistic Regression model to predict the class outcome.

The project solves the problem of providing a simple computer-based prediction interface for a diabetes dataset. It combines data exploration, visualization, preprocessing, machine learning, model evaluation and a Flask web interface in one project.

The system is useful for educational demonstration of how AI/ML classification can be integrated into a web application. It is not a medical diagnostic tool and its output should not be used as a substitute for professional medical assessment.

2. Problem Statement :

To develop an AI-based web application that analyzes selected health-related attributes from a diabetes dataset and predicts whether the input belongs to the negative or positive outcome class using a trained machine learning classification model.

3. Objectives :

- To load and explore the diabetes dataset using Python.
- To preprocess the dataset by handling impossible zero measurements and preparing the features for machine learning.
- To train and test a Logistic Regression classification model.
- To evaluate the model using accuracy, precision, recall, F1 score and a confusion matrix.
- To develop a Flask-based web interface that accepts user inputs and displays the model prediction.

4. Technologies Used :

Technology / Tool	Purpose
Python	Programming language used for data processing, machine learning and backend development.
Pandas	Dataset loading, manipulation and analysis.
NumPy	Numerical data processing used by the ML workflow.
Matplotlib	Data visualization.
Seaborn	Statistical visualization and correlation heatmap.
Scikit-learn	Preprocessing, Logistic Regression, train/test split and model evaluation.
Joblib	Saving and loading the trained machine learning pipeline.
Flask	Web application backend and form handling.
HTML/CSS	User interface for entering prediction inputs and viewing results.
VS Code	Development environment.

5. Dataset :

Dataset Name: diabetes.csv

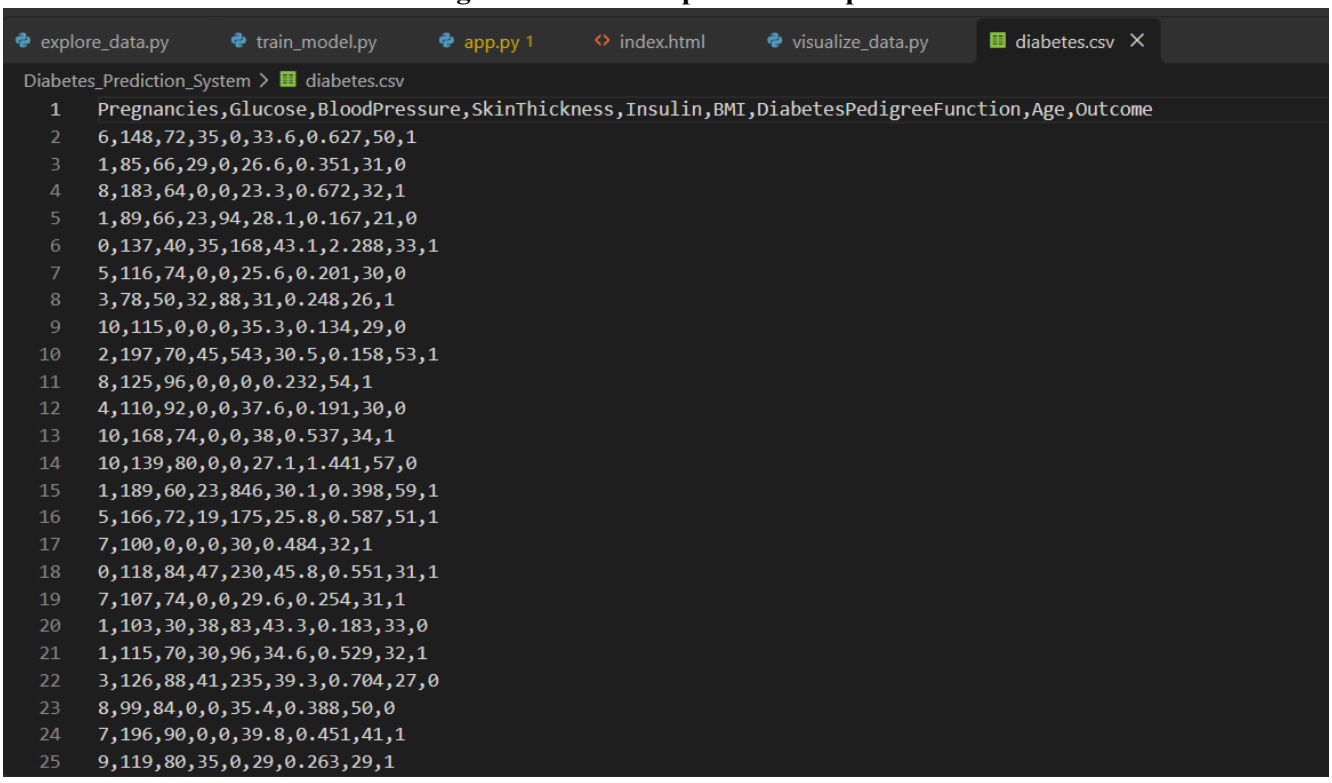
The project dataset contains 768 records and 9 columns. Eight columns are input features and the ninth column, Outcome, is the target variable. The project files do not specify an external source URL for the CSV, so the dataset source is recorded here as the diabetes.csv dataset supplied with the project.

Attribute	Description / Role
Pregnancies	Number of pregnancies.
Glucose	Glucose measurement.
BloodPressure	Blood pressure measurement.
SkinThickness	Skin thickness measurement.
Insulin	Insulin measurement.
BMI	Body Mass Index.
DiabetesPedigreeFunction	Diabetes pedigree function value.
Age	Age of the person.
Outcome	Target variable: 0 = Negative, 1 = Positive.

Dataset summary:

- Total records: 768
- Total columns: 9
- Input features: 8
- Target variable: Outcome
- Outcome 0 records: 500
- Outcome 1 records: 268

Figure 1: Dataset exploration output .



6. Methodology / Working :

The project follows the steps below:

- Load the diabetes.csv dataset using Pandas.
- Explore the first records, shape, columns, statistical summary, missing values and outcome distribution.
- Visualize the outcome distribution and feature correlation heatmap.

- Separate the dataset into input features (X) and target variable (y).
- Replace impossible zero measurements in Glucose, BloodPressure, SkinThickness, Insulin and BMI with missing values.
- Split the data into 80% training data and 20% testing data using stratification.
- Build a preprocessing and classification pipeline using median imputation, StandardScaler and Logistic Regression.
- Train the model on the training data.
- Predict outcomes for the testing data.
- Evaluate the model using accuracy, precision, recall, F1 score, confusion matrix and classification report.
- Save the complete trained pipeline as diabetes_model.pkl.
- Load the model in the Flask application, accept user input, validate it and display the prediction.

6.1 Project Workflow :

diabetes.csv

↓

Data Exploration

↓

Data Visualization

↓

Data Preprocessing

↓

Train/Test Split

↓

Logistic Regression Model

↓

Model Evaluation

↓

diabetes_model.pkl

↓

Flask Web Application

↓

Prediction Result

7. Algorithm Used :

7.1 Logistic Regression

Logistic Regression is a supervised machine learning classification algorithm used to predict the probability of a categorical outcome. In this project, it is used for binary classification of the Outcome variable into two classes: 0 (negative) and 1 (positive).

The model first receives the preprocessed feature values. StandardScaler standardizes the features, and Logistic Regression learns the relationship between the input variables and the target class. The trained model then predicts the class for new input values.

Logistic Regression was selected because the project requires binary classification and the algorithm is simple, interpretable and suitable for a small structured dataset.

7.2 Preprocessing Used

- SimpleImputer with median strategy is used to fill missing values.
- StandardScaler is used to standardize numerical features.
- LogisticRegression(max_iter=1000) is used as the classifier.
- The complete preprocessing and model steps are stored together in a Scikit-learn Pipeline.

8. Implementation

The implementation is divided into four Python programs. The code below is the actual code used in the submitted project.

8.1 explore_data.py

```
# Diabetes Prediction System
# Step 1: Load and explore the dataset

import pandas as pd

df = pd.read_csv("diabetes.csv")

print("First five rows:")
print(df.head())

print("\nDataset shape:")
print(df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe())

print("\nMissing values:")
print(df.isnull().sum())

print("\nOutcome distribution:")
print(df["Outcome"].value_counts())
```

Purpose: Loads the CSV and displays sample records, dataset shape, columns, information, statistical summary, missing values and outcome counts.

8.2 visualize_data.py

```
# Diabetes Prediction System
# Step 2: Visualize the dataset

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("diabetes.csv")
```

```
# Outcome distribution
sns.countplot(data=df, x="Outcome")
plt.title("Diabetes Outcome Distribution")
plt.xlabel("Outcome (0 = Negative, 1 = Positive)")
plt.ylabel("Number of Records")
plt.tight_layout()
plt.show()

# Correlation heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Feature Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Purpose: Generates the diabetes outcome distribution chart and the feature correlation heatmap.

Figure 2: Diabetes Outcome Distribution graph .

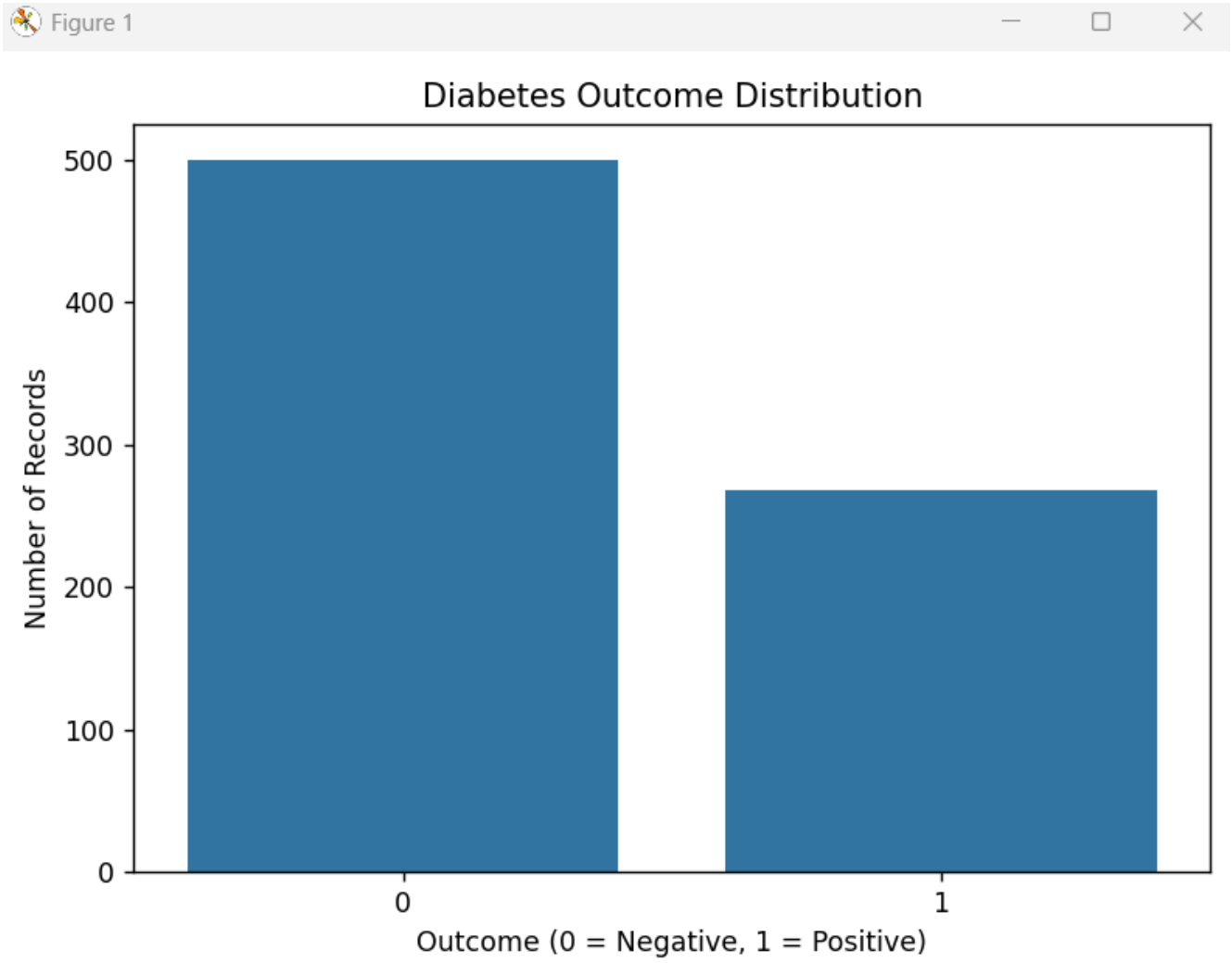
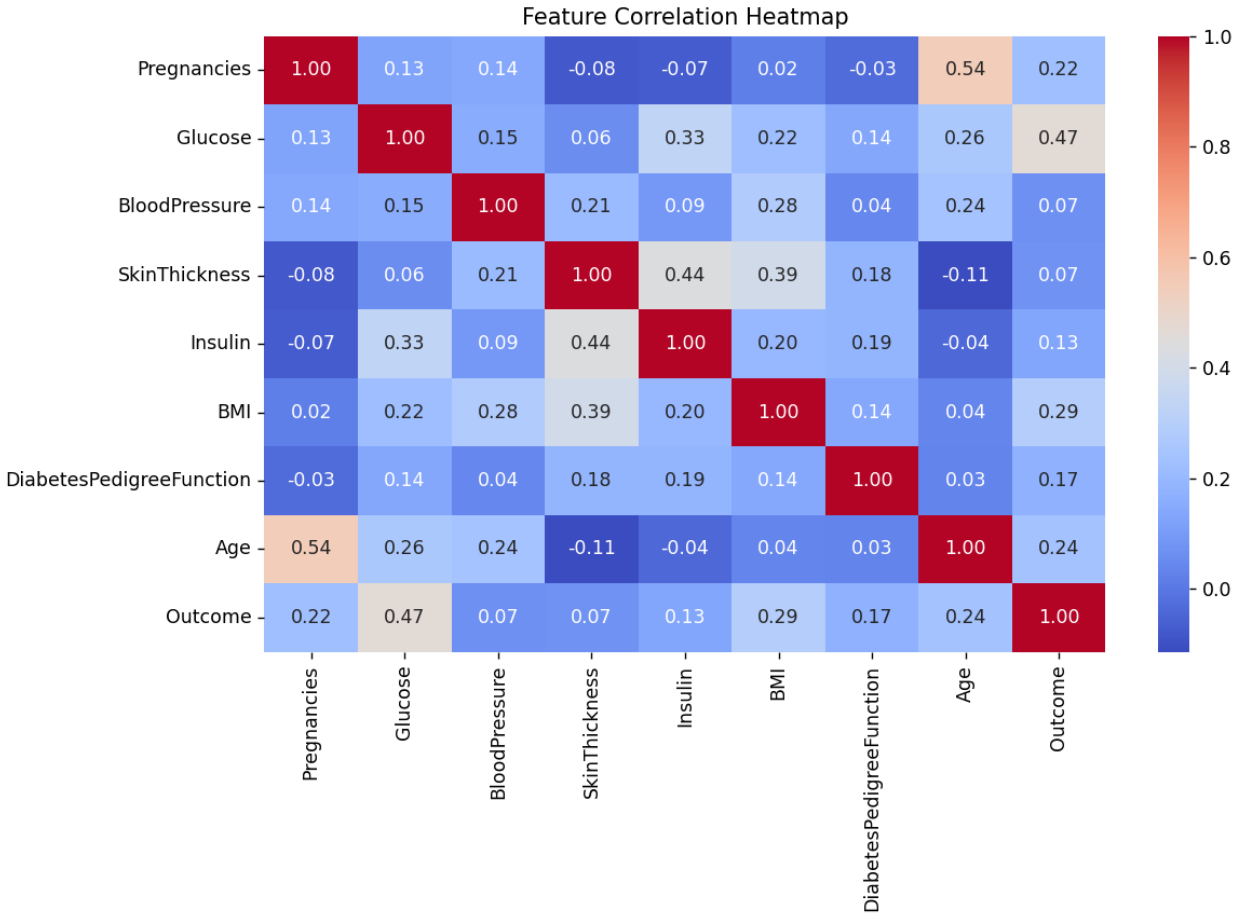


Figure 3: Feature Correlation Heatmap .



8.3 train_model.py

```
# Diabetes Prediction System
# Step 3: Train and evaluate the Machine Learning model

import pandas as pd
import joblib

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# 1. Load dataset
df = pd.read_csv("diabetes.csv")

# 2. Separate features and target
X = df.drop("Outcome", axis=1)
```

```
y = df["Outcome"]

# 3. Replace impossible zero measurements with missing values
zero_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

X[zero_columns] = X[zero_columns].replace(0, float("nan"))

# 4. Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 5. Create a preprocessing and model pipeline
model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])

# 6. Train the model
model.fit(X_train, y_train)

# 7. Predict on test data
y_pred = model.predict(X_test)

# 8. Evaluate the model
print("Model Evaluation Results")
print("------------------------")
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(
    precision_score(y_test, y_pred, zero_division=0), 4
))
print("Recall:", round(
    recall_score(y_test, y_pred, zero_division=0), 4
))
print("F1 Score:", round(
    f1_score(y_test, y_pred, zero_division=0), 4
))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# 9. Save the complete trained pipeline
joblib.dump(model, "diabetes_model.pkl")

print("\nModel saved as diabetes_model.pkl")
```

Purpose: Performs preprocessing, train/test splitting, model training, evaluation and saves the trained pipeline as diabetes_model.pkl.

Figure 4: Model training and evaluation console output.

```
PS C:\Users\prana\OneDrive\Desktop\AI\Project\Diabetes_Prediction_System> python train_model.py
Model Evaluation Results
------------------------
Accuracy: 0.7078
Precision: 0.6
Recall: 0.5
F1 Score: 0.5455

Confusion Matrix:
[[82 18]
 [27 27]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.82      0.78       100
           1       0.60      0.50      0.55        54

    accuracy                           0.71       154
   macro avg       0.68      0.66      0.67       154
weighted avg       0.70      0.71      0.70       154


Model saved as diabetes_model.pkl
```

8.4 app.py

```
# Diabetes Prediction System
# Step 4: Flask backend

from flask import Flask, render_template, request
import pandas as pd
import joblib

app = Flask(__name__)

# Load trained model
model = joblib.load("diabetes_model.pkl")

FEATURES = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age"
]


@app.route("/", methods=["GET", "POST"])
def home():
    result = None
    error = None
    values = {}

    if request.method == "POST":
```

```
        try:
            # Get form values
            for feature in FEATURES:
                values[feature] = float(request.form[feature])

            # Basic validation
            if any(value < 0 for value in values.values()):
                raise ValueError("Values cannot be negative.")

            if values["Age"] <= 0:
                raise ValueError("Age must be greater than zero.")

            if values["BMI"] <= 0:
                raise ValueError("BMI must be greater than zero.")

            if values["Glucose"] <= 0:
                raise ValueError("Glucose must be greater than zero.")

            # Prepare input in the same column order as training
            input_data = pd.DataFrame(
                [[values[f] for f in FEATURES]],
                columns=FEATURES
            )

            # Predict class
            prediction = int(model.predict(input_data)[0])

            if prediction == 1:
                result = (
                    "Positive class predicted by the model. "
                    "This is not a diagnosis; please consult "
                    "a qualified healthcare professional."
                )
            else:
                result = (
                    "Negative class predicted by the model. "
                    "This does not rule out diabetes. "
                    "Consult a healthcare professional if concerned."
                )

        except (ValueError, KeyError):
            error = "Please enter valid numerical values in all fields."

    return render_template(
        "index.html",
        result=result,
        error=error,
        values=values
    )


if __name__ == "__main__":
    app.run(debug=True)
```

Purpose: Runs the Flask web application, validates form values, prepares the input DataFrame, loads the trained model and displays the prediction result.

Figure 5: Diabetes Prediction System web interface .

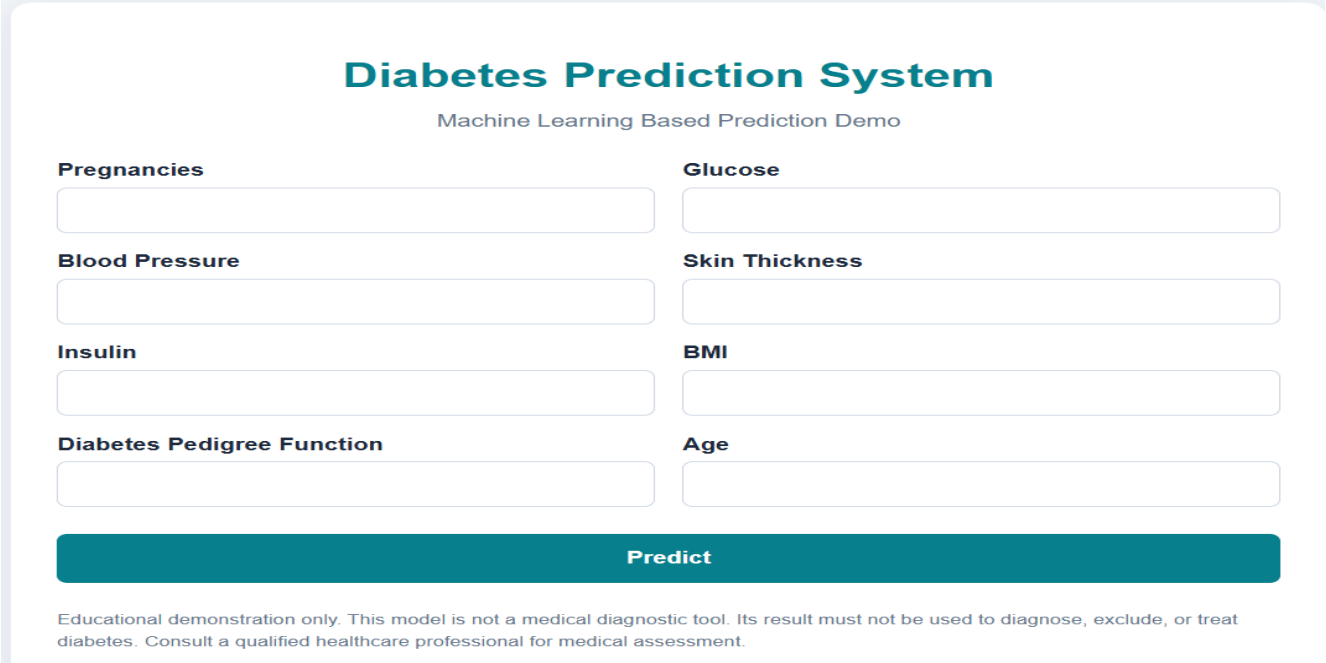
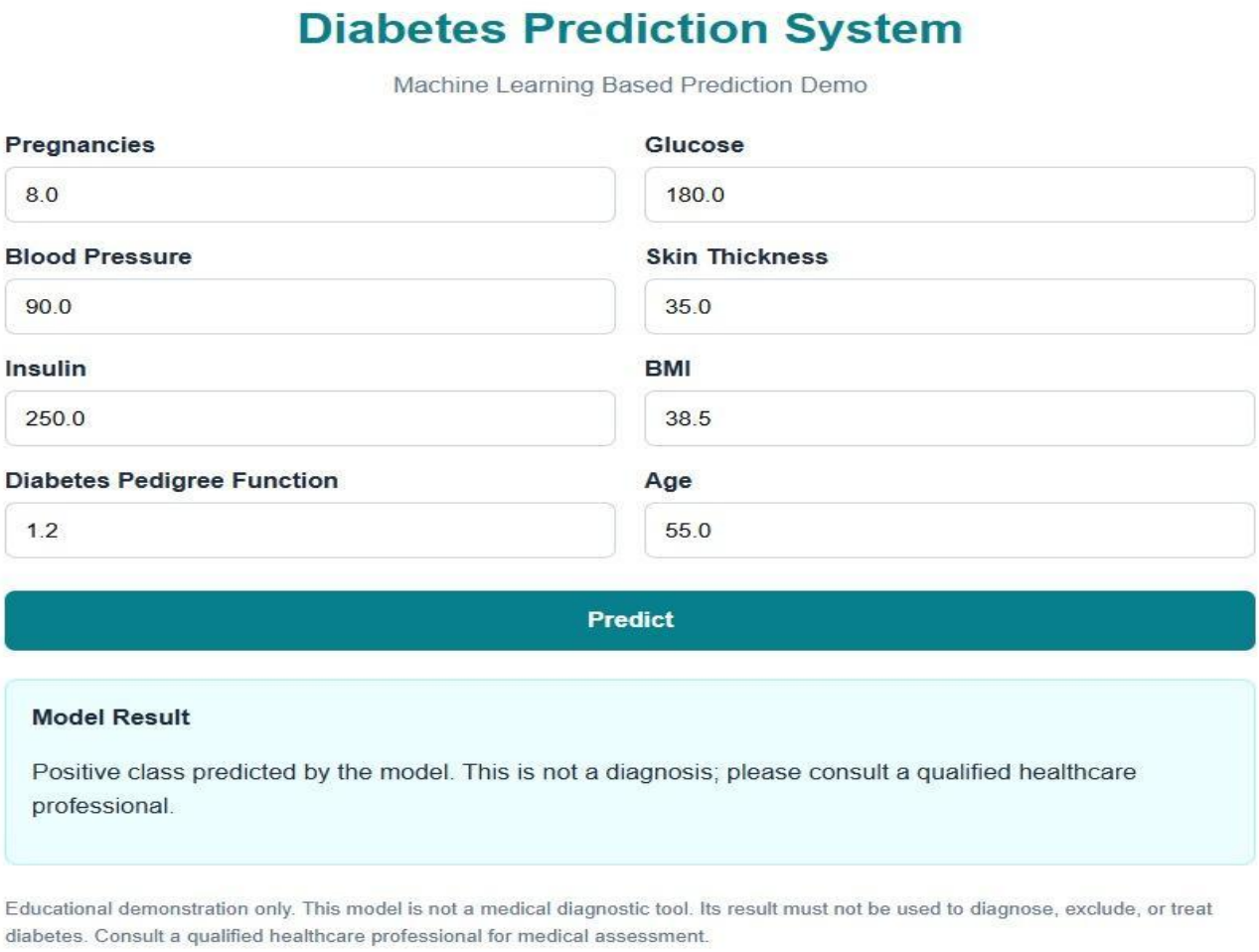


Figure 6: Prediction result/output screenshot .



9. Results :

The trained Logistic Regression model was evaluated on the 20% test set. The test set contains 154 records. The model achieved the following performance:

Metric	Result
Accuracy	70.78%
Precision	60.00%
Recall	50.00%
F1 Score	54.55%

Confusion Matrix:

	Predicted 0	Predicted 1
Actual 0	82	18
Actual 1	27	27

The confusion matrix shows 82 correctly predicted negative cases, 27 correctly predicted positive cases, 18 negative cases predicted as positive and 27 positive cases predicted as negative.

10. Advantages :

- Provides a simple web-based interface for entering health parameters.
- Uses an automated machine learning model for binary classification.
- Includes data preprocessing before model training.
- Provides multiple evaluation metrics to assess model performance.
- The trained model is saved and can be reused by the Flask application.

11. Limitations :

- The model is trained on a limited dataset of 768 records.
- The achieved accuracy is about 70.78%, so predictions are not perfectly accurate.
- The system is an educational prediction project and is not a medical diagnostic tool.
- The application currently uses a single Logistic Regression model rather than an ensemble or deep learning comparison.

12. Applications :

- Educational demonstration of AI/ML classification.
- Machine learning laboratory and academic mini-projects.
- Demonstration of integrating a trained ML model with a web application.
- Prototype for health-risk analytics and data science learning.

13. Conclusion :

The AI-Based Diabetes Prediction System was successfully developed as a machine learning web application. The project loads and explores a diabetes dataset, performs preprocessing, trains a Logistic Regression classifier, evaluates its performance and saves the trained pipeline for use in a Flask application.

The final system provides a user-friendly form where eight input features can be entered and the trained model returns a negative or positive class prediction. The project demonstrates the complete workflow of an AI application, from dataset preparation and model training to web-based prediction. The result is intended only for educational purposes and not for medical diagnosis.

14. Future Scope :

- Compare Logistic Regression with Decision Tree, Random Forest, KNN and other classifiers.
- Use hyperparameter tuning and cross-validation to improve model performance.
- Add probability/confidence display and more detailed analytics.
- Add user authentication and prediction history.
- Create a database to store prediction records.
- Improve the frontend with a dashboard and additional visualizations.
- Deploy the application to a cloud hosting platform for controlled educational use.
- Use a larger and more diverse dataset for better generalization.

15. References :

- Scikit-learn documentation – Logistic Regression, preprocessing, pipelines and model evaluation.
- Pandas documentation – Data loading and data analysis.
- Matplotlib documentation – Data visualization.
- Seaborn documentation – Statistical data visualization.
- Flask documentation – Python web application development.
- Joblib documentation – Model serialization and loading.
- Project dataset: diabetes.csv supplied with this mini project. The uploaded project files do not specify an external dataset URL.

16. GitHub Repository :

GitHub Link https://github.com/SiddheshK-04/AI-Diabetes-Prediction-System-AI-Mini-Project